# <World Happiness Index>



Kaplan Business School

A report submitted in partial fulfillment of the requirements for the <Bachelor of Information Technology>

**Team Members**

*<Rohan Sharma - 1860694>*

*<Varaj Chetankumar Patel>*

*<Yash Sharma>*

**MONTH YEAR- June 2026**

## Abstract

*The project is based on the world happiness report dataset from 2015 to 2019. The main purpose of the software is to help users understand and analyze happiness data from different countries in an easy way. During the development of this software my teammates and I do not only learn the technical expectations of the coding but also get familiar with terms like GDP, social support, life expectancy, freedom, generosity and corruption in relation to a country's happiness. The software we develop allows users to specifically select a year and then train a machine learning model using the data from that year. Users can view analysis results, generation of charts and predictions of happiness based on the selected year - dataset. Also, it handles the variations if a user wants to use its own values for GDP, social support, etc. it can predict and give results according to user values as well.*

## Research about the problem and project overview

The dataset we have allocated has 5 CSV form the world happiness report between 2015 to 2019. The files contain attributes such as country, region, happiness rank, happiness score, standard error, economy (GDP per capita), Health (life expectancy), Freedom, Trust (Government Corruption), Generosity, Dystopia residual.

The problem is that trends and patterns are difficult to identify from raw numbers alone. We began by asking ourselves how this dataset could be used to create software which can help a user to get an overview of a dataset and improve their understanding about it.
During the discussion we considered several questions. What could be the simplest and easiest comparisons that can be made using the dataset? For example, the top 10 happiest countries, the top saddest countries. Which variables have stronger relationship with happiness scores compared to some other factors We first created a meaning for us what could be a possible software that can be made using this dataset by writing some questions for us and by reading the content given on the keggele website.

We also explored whether happiness scores could be estimated using the available factors. Since the dataset contains both the happiness score and the contributing factors, we decided to include a machine learning component in our software. This allowed users to enter factor values and obtain an estimated happiness score based on patterns learned from the selected year's data.

A table of values does not clearly show whether happiness is increasing, decreasing, or how different factors are affecting each other. Therefor Visual charts, summaries and mathematical calculations have been included in the software to make the information easier to understand and interpret

## 1. Project Overview

1.1. **Project Description –** The world happiness report analyser is a desktop application developed using python programming language. The software combines data analysis, data visualisation, and machine learning features with a single graphical user interface.

### 1.2. Objectives -

To provide user an easy way to analyse the dataset.
To help user to find trends and patterns in the data.
To allow comparisons of happiness scores across different countries and years.
To apply machine learning techniques on selected variables.
To develop a user-friendly application using python.

1.3. **Scope -** The scope of dataset is limited to given dataset from 2015 to 2019. We first decided to allow user to manually load dataset, but that feature was making our coding part complex and there were high chances of errors. The application does not collect real time data or connect to external databases or provide information beyond the

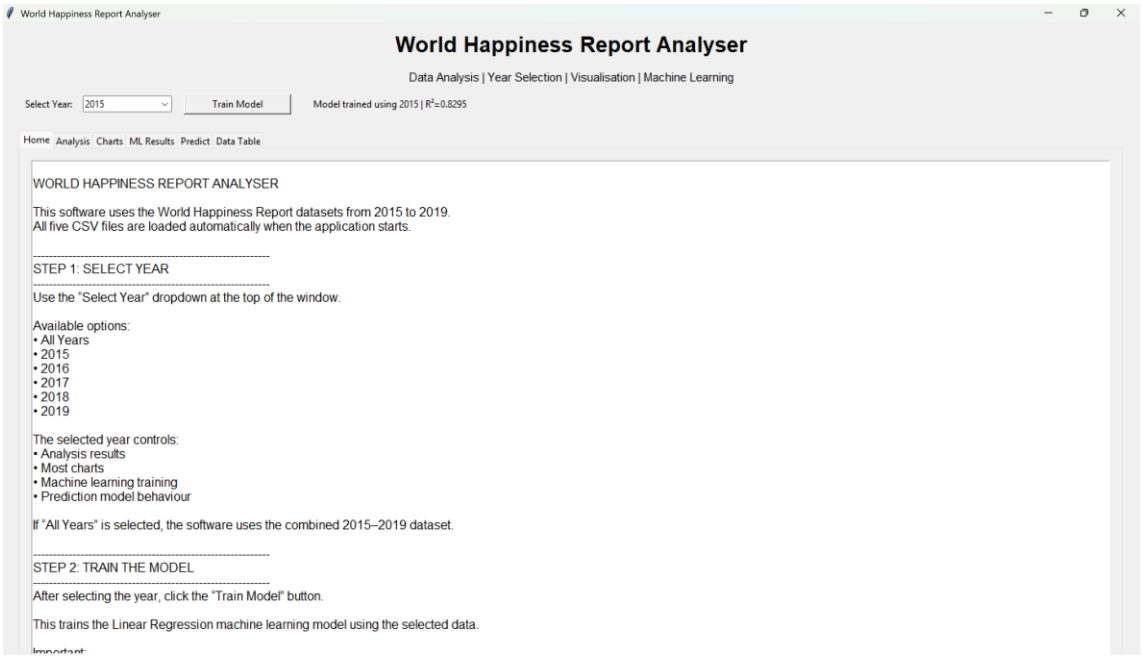
information contained within the supplied datasets. The prediction function is given to make the software interactive for the user.

## 2. Software Details

2.1. **Technology Stack –** Python consists of wide range of libraries for data analysis, visualisation, machine learning and graphical user interface development.
Following libraries have been used during development
Tkinter – Used to create the graphical interface (GUI), including tabs, buttons, dropdown menus, labels and input fields.
Pandas – Used to load, clean, filter and analyse the dataset.
Matplotlib – Used to generate charts and visualise trends and realtionships within the dataset.
Scikit – learn – Used to implement the liner regression machine learning model for predicting happiness scores.

2.2. **Architecture** The application is written in a single Python program. The software consists of three main parts. First is the graphical user interface, which allows users to interact with the software through different tabs and options. Second is the data processing component, which uses Pandas to load, clean, filter, and analyse the datasets. Third is the machine learning component, which uses Scikit-learn to train a Linear Regression model and predict happiness scores based on factors such as GDP, social support, life expectancy, freedom, generosity, and corruption.

2.3. **Features –** The goal was to create a software which is user friendly and efficient to calculate and read data in different ways with different comparison.




Tabs created in the software and how they work:

Home tab – It is a built-in user guide. It explains the purpose of the software and provides step by step instructions on how to use the different functions. This help user to understand the workflow of the application and the do's and don'ts before performing analysis predictions.

Select year – This option allows user, to read and analyse a Specific year by training the program with selected dataset.

```
    def train_model(self):
        """Train the Linear Regression model using selected data."""

        selected_data, year_used = self.get_selected_data()

        available_features = [f for f in FEATURES if f in selected_data.columns]

        if len(selected_data) < 10:
            messagebox.showerror( title: "Error",  message: "Not enough data to train model.")
            return

        X = selected_data[available_features]
        y = selected_data["Score"]

        # Split data into training and testing sets.
        X_train, X_test, y_train, y_test = train_test_split(
             *arrays: X,
            y,
            test_size=0.2,
            random_state=42
        )
```

**Figure shows training of dataset**

Analysis section consists of three options -

The dataset summary provides an overview of the selected dataset. It displays information such as the total number of records, total number of countries, average happiness scores, and descriptive statistics. When all years are selected, the software displays the average happiness score for each year.

Top 10 countries feature identifying and displaying countries with the highest happiness score based on the selected year. Similarly, the lowest countries button displays the countries with the lowest happiness scores for the selected year. This allows the user to compare and see the trends in happiness of countries in different years.

Chart tab – Provides visual graphs for the above features including Average happiness from all years, GDP vs happiness( scatter plot is used where bottom left is countries with low GDP , low happiness and top right is countries with high GDP and high happiness looking at graph we can say that as GDP increases happiness score generally increases, this is called a positive correlation.

**ML result -** The ML Results section displays the performance of the Linear Regression model after training. It shows the number of training and testing records, prediction accuracy measures (MAE, RMSE, and $R^2$), and the coefficients for each factor used in prediction. The coefficients indicate the relative influence of GDP, social support, life expectancy, freedom, generosity, and corruption on the predicted happiness score. The results help users understand both the accuracy of the model and the relationship between different factors and happiness levels.

Predict section allows the user to find the happiness score using the linear regression formula:

$y=b0 +b1 (GDP)+b2 (Social Support)+b3 (Life Expectancy)+b4 (Freedom)+b5 (Generosity)+b6 (Corruption)$

$y$ = Predicted Happiness Score
$b_o$ = Intercept

$b_1$–$b_6$ = Learned coefficients from the trained model

## 3. Development Process

3.1. **Challenges -** One challenge is deciding which attributes should be include in the analysis and prediction model. A lot of time has been spent studying the structure and purpose of variable such as GDP, social support, life expectancy, freedom, generosity and corruption.

Working with datasets from different year was also challenging. Although datasets contained similar information column names were different across the CSV files. This required dataset standardisation process before the dataset could be combined and analysed consistently.

```
49  COLUMN_RENAME = {
50      "Country or region": "Country",
51      "Country": "Country",
52
53      "Happiness Rank": "Rank",
54      "Happiness.Rank": "Rank",
55      "Overall rank": "Rank",
56
57      "Happiness Score": "Score",
58      "Happiness.Score": "Score",
59      "Score": "Score",
60
61      "Economy (GDP per Capita)": "GDP",
62      "Economy..GDP.per.Capita.": "GDP",
63      "GDP per capita": "GDP",
64
65      "Family": "Social Support",
66      "Social support": "Social Support",
67
68      "Health (Life Expectancy)": "Life Expectancy",
69      "Health..Life.Expectancy.": "Life Expectancy",
70      "Healthy life expectancy": "Life Expectancy",
71
72      "Freedom": "Freedom",
73      "Freedom to make life choices": "Freedom",
74
75      "Generosity": "Generosity",
```

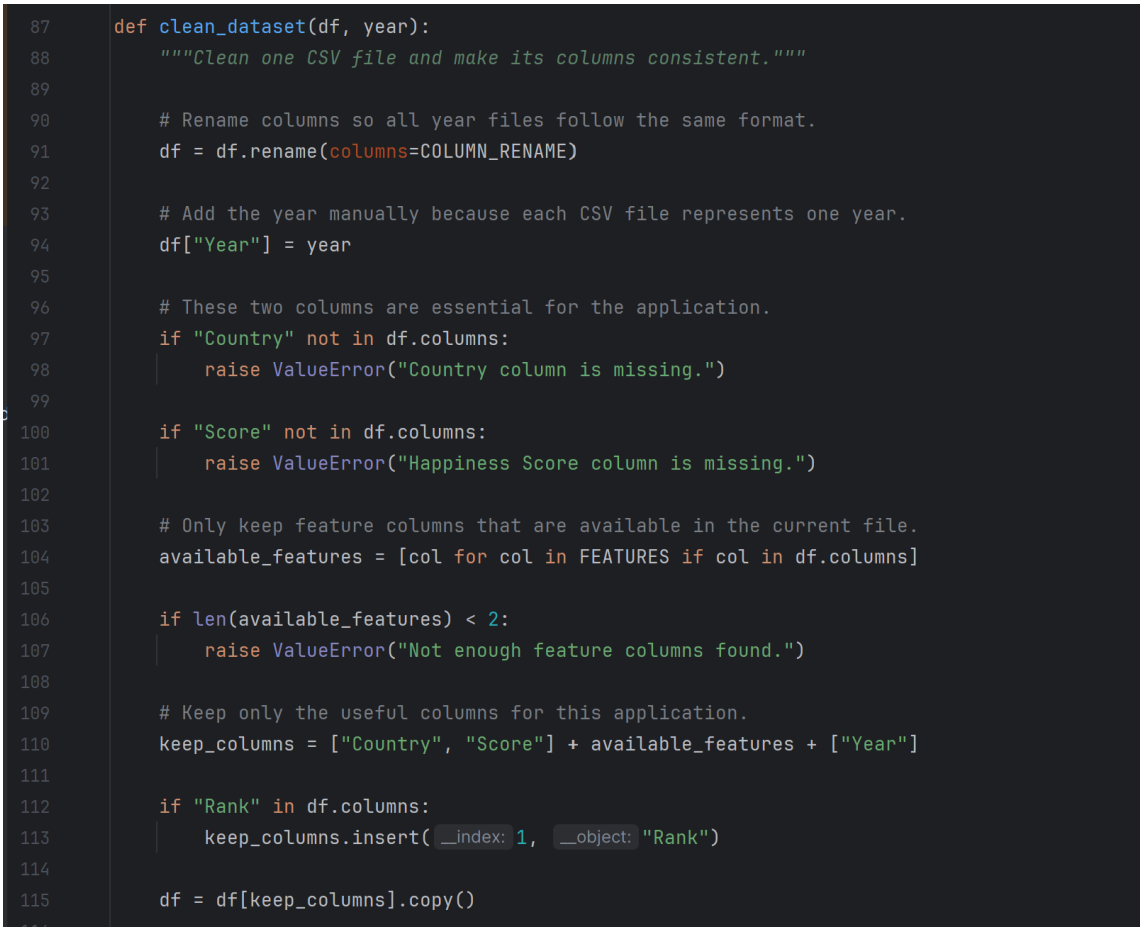
**Figure shows data standardisation.**

Another challenge was that there are too many attributes and metrics like MAE, RMSE, and $R^2$ that need to be followed to provide meaningful understanding of datasets. We created the home tab so that users can understand how to use the software and make it possible to show them what we exactly want to show.

Different approaches have been discussed first we decided to allow manual uploading of datasets. However, difference in dataset structure could make our code inconsistent and introduce as we need similar files to make a clear objective of the software.

3.2. **Code Quality -** Code is written in a reliable organised manner. The program was divided into functions, each responsible for a specific task. For example, load_all_data() was used load the datasets m clean_dataset() was used to clean and standardise the data and get_selected_data() was used to retrieve data based on the year selectd by the user.

Separate functions were also created for the main features of the software. Basic error handling was included to prevent common issues to check whether all required CSV files are available before loading data and prevents users from making predictions before training the machine learning model.



```python
def clean_dataset(df, year):
    """Clean one CSV file and make its columns consistent."""

    # Rename columns so all year files follow the same format.
    df = df.rename(columns=COLUMN_RENAME)

    # Add the year manually because each CSV file represents one year.
    df["Year"] = year

    # These two columns are essential for the application.
    if "Country" not in df.columns:
        raise ValueError("Country column is missing.")

    if "Score" not in df.columns:
        raise ValueError("Happiness Score column is missing.")

    # Only keep feature columns that are available in the current file.
    available_features = [col for col in FEATURES if col in df.columns]

    if len(available_features) < 2:
        raise ValueError("Not enough feature columns found.")

    # Keep only the useful columns for this application.
    keep_columns = ["Country", "Score"] + available_features + ["Year"]

    if "Rank" in df.columns:
        keep_columns.insert( _index: 1,  _object: "Rank")

    df = df[keep_columns].copy()
```

**Figure shows data cleaning and validation function.**

4. **Results and Outcomes – The** software successfully combined the CSV files and performed desired tasks. Machine learning models can generate the happiness score predictions. The charts and analysis feature also helped transform large amounts of raw data into information that is easier to understand.

5. **Learning and Conclusion:** Through this project I learned the importance of communication between teammates. Understanding each other's ideas helped us develop this software. Additionally, I deeply understand the Python libraries, gained experience with datasets that contain different structures across multiple files and learned how standardization can help create a consistent data analysis process.

This project improved both my programming skills and my understanding of data analysis and machine learning concepts.